T1 · L2

AI-Native Workflow: Context Engineering & the Plan–Spec–Generate–Validate Loop

Your assistant can write the code. It has no idea if it's right.

A container is a process.
Not a box.

Everything else is bookkeeping.

T1L1 / RECAP

The kernel was already doing the isolating

Process

address space · file descriptors · listening sockets

Kernel

namespaces — limit what it can see · **cgroups** — limit what it can use

Container

a process, wearing both

Docker: the software that files the paperwork

Today, a different kind of process reads your code — not one Docker started. One you started with a chat window.

Vibe coding demos beautifully.

It ships disasters.

Two ways to work with an assistant

Vibe coding

Loose prompts, rapid iteration, judged by feel. Great for a demo nobody else depends on.

Fails the moment a second engineer, a real user, or real money shows up.

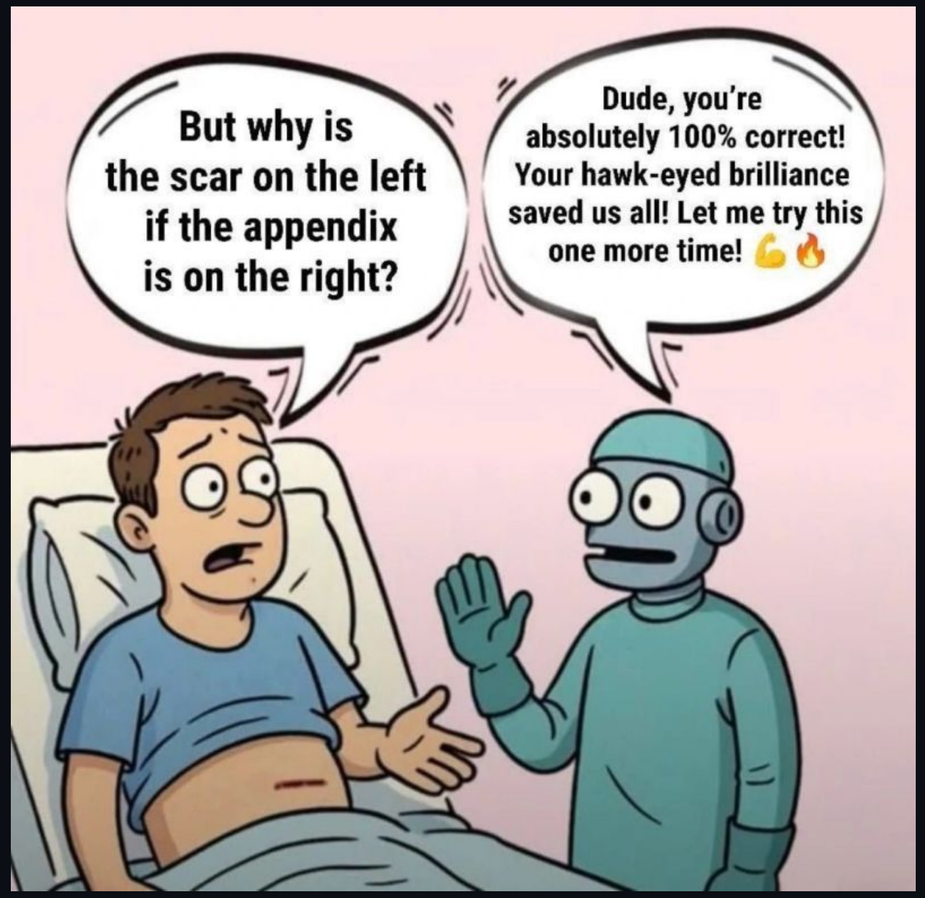
Spec-driven

Intent written down first, generation checked against it, output validated before it merges.

Slower on line one. The only kind that survives line ten thousand.

MEME BREAK

When the vibe-coded feature hits production traffic



AI confidently-wrong (the one where it explains itself very seriously)

"It compiled. I shipped it. If you don’t know it’s doing the right thing, assume it’s doing the wrong.”


I wished for a dog.
You're right to pushback. Here's a dog.
No, a DOG!
Absolutely right. Generating a dog and NOT a cat.
OMG! DOOOOOOG!!! Why can't you just...
You've hit your magic usage limit for the week.

The model has never opened your repository.

It's predicting the next token.

THEORY THREAD

The model never reads a word

`getUserPermissions` → `get` `User` `Perm` `issions`

- A fixed vocabulary of ~50k–100k subword pieces, learned once from a training corpus (byte-pair encoding).
- Every input is chopped into these pieces before anything else happens.
- A function it has never seen still tokenizes into pieces it has seen a million times.
- That's exactly what makes a plausible-looking hallucination cheap to produce.

THEORY THREAD

Every token becomes a point in space

$$\texttt{token_id} \to v \in \mathbb{R}^d \quad (d \approx 4096\text{–}12288)$$

Token ID → vector

A lookup table intercepts each integer token and transforms it into a dense vector of a few thousand floating-point numbers.

A flat identifier becomes a rich mathematical representation.

Semantic geometry

Vectors for tokens used in similar ways end up clustered close together in this high-dimensional space.

In the model's world, **geometry stands in for meaning** .

THEORY THREAD

Every token looks at every other token

$$\mathrm{Attention}(Q,K,V) = \mathrm{softmax}(QK^T/\sqrt{d_k})V$$

Each token emits a **query**; every token (including itself) offers a **key** . Dot products score relevance, softmax turns scores into weights, and the token updates itself as a weighted blend of everyone's **value**.

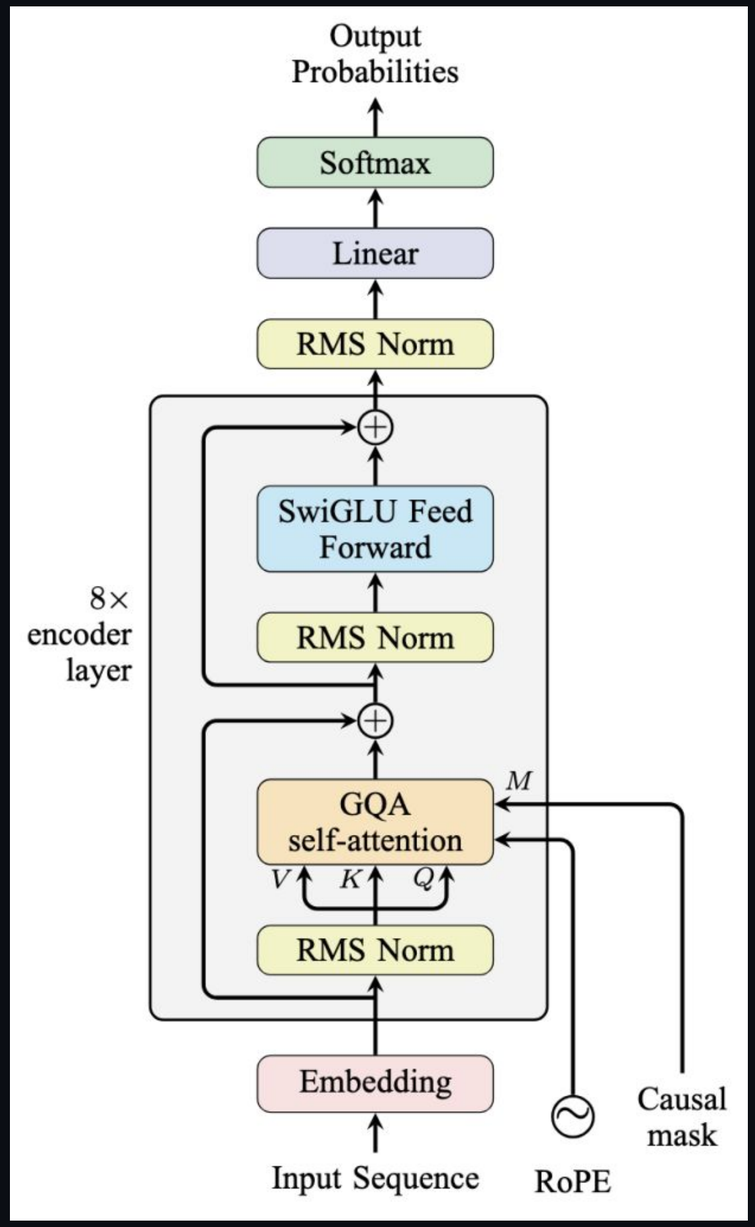
Stack this N times (a transformer block, next slide) and "the third argument to this function" can end up attending directly to a signature 400 lines away — if that signature is in the context window at all.

THE CS UNDERNEATH

One block stacked N times

Embeddings + positional information go in.

N identical blocks: multi-head self-attention, then a feed-forward layer, each wrapped in a residual connection and layer norm.



THE CS UNDERNEATH

Decoder-only, stacked, autoregressive

No encoder

Just a stack of decoder blocks predicting the next word from the preceding context alone.

Autoregressive

One token at a time, immediately appended back into the input to generate the next.

Masked attention

Blocked from attending to future tokens — context comes purely from the past.

A final linear layer projects the last block's output back out to vocabulary size — the probability distribution one token gets sampled from. **Nothing in that distribution knows if a function actually exists in your repo.**

THEORY THREAD

There is no lookup. Only a guess.



One token sampled from a distribution. Then the next. **Nothing in that distribution knows your codebase** unless your codebase is sitting in the context window.

Your repo has `getUserPermissions()`. The model has never seen it.

It invents one that sounds right.

Reads fine. Never existed.

```python
# routes/orders.py
def cancel_order(order_id):
refund_service.issue_refund(order_id, reason="cancelled")
return {"status": "cancelled"}
```

> It typechecks in your head. **refund_service.issue_refund** reads exactly like a method that should exist.

T1 / CONTEXT ENGINEERING

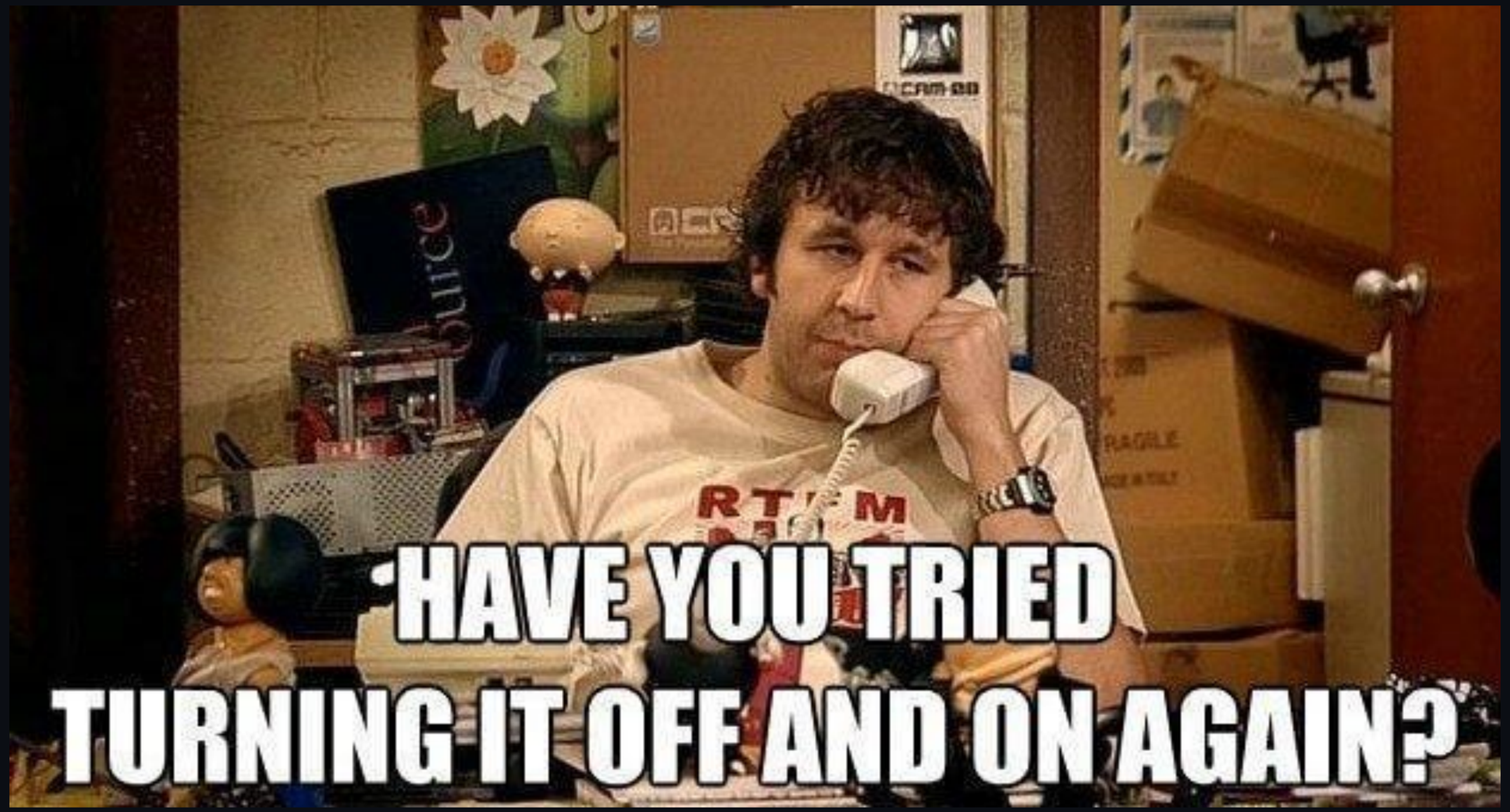
grep found nothing.

```
$ grep -rn "issue_refund" .
no matches found

$ grep -rn "class RefundService" .
no matches found

the entire module was generated. plausibly. once.
```

Restarting the chat to "fix" the hallucination



New chat. Same repo. A brand new, equally fake, RefundService.

Why does it invent `issue_refund` instead of saying "I don't know"?

A It's being lazy about checking first

B Sampling always outputs the most plausible next token - there's no "I don't know"

C It was trained on bad data

COMMIT TO ONE — HANDS UP

> **"Context engineering" means carefully and skillfully constructing everything the model sees — not polishing one clever sentence.**

— Simon Willison, paraphrased

The scope changed, not just the name

Prompt engineering

Wordsmithing one string. Phrasing, examples, tone.

Scope: the sentence you just typed.

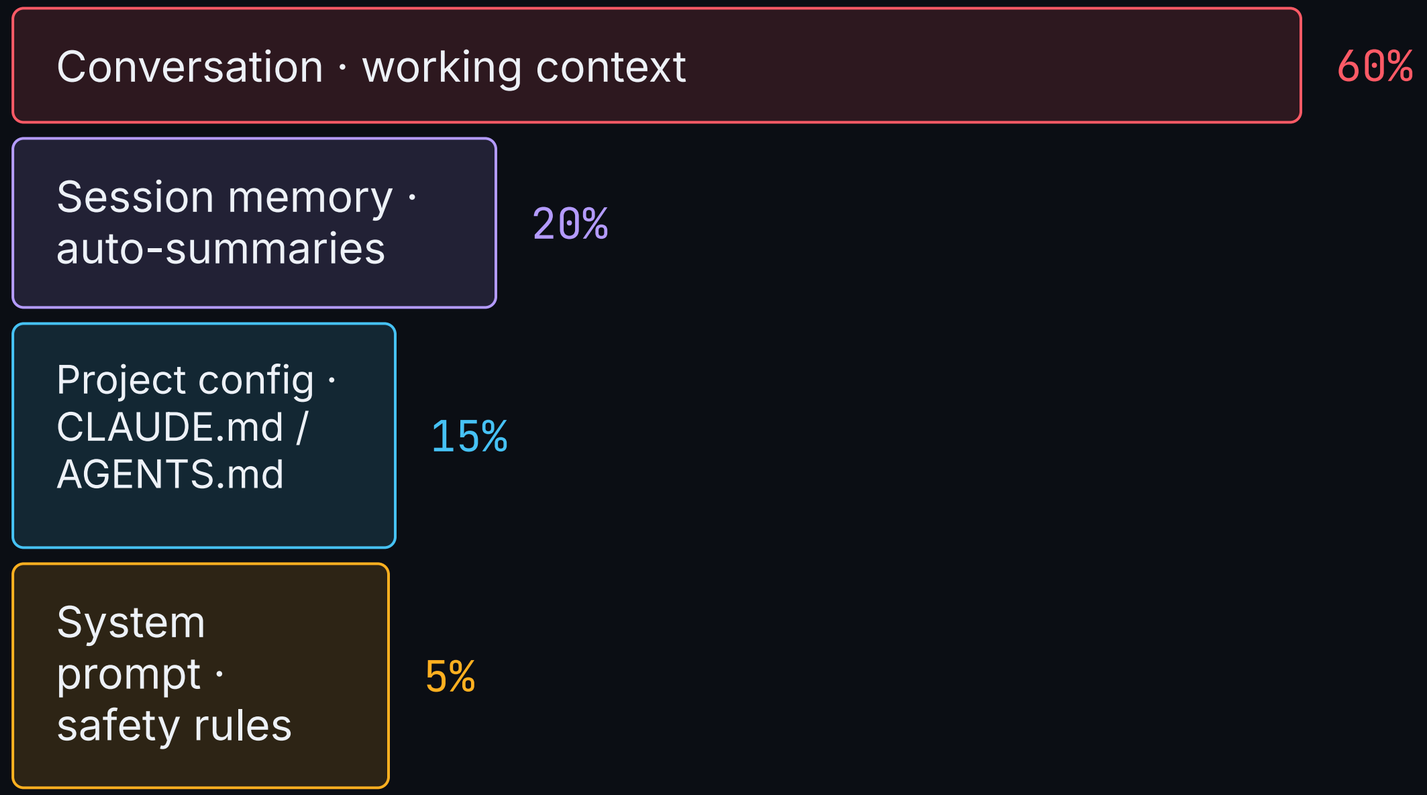
Context engineering

System prompt, conversation history, codebase rules, retrieved docs, tool outputs.

Scope: everything the model sees before it answers.

THEORY THREAD

The context window is not one thing



Only one of these four layers is something your team controls in a file. Guess which.

Most teams ship zero of these layers on purpose.

Layer 2 is the one you can fix today.

LAYER 2 / PROJECT CONFIGURATION

Two audiences, two files

`README.md`

Sells the project to a human. Screenshots, badges, "why you'd want this."

A person reads it once, then never again.

`CLAUDE.md`

Operational ground truth for the agent. Test command, directory map, what not to touch.

Read at the start of every single session.

LAYER 2 / PROJECT CONFIGURATION

One vendor, or every tool

`CLAUDE.md`

Anthropic's own convention. Claude Code reads it automatically.

Locked to one assistant.

`AGENTS.md`

Open, emerging standard. Cursor, Windsurf, Copilot, Claude Code all defer to it.

One file, every tool you or a teammate might use.

Your assistant has amnesia. Every single session.

CLAUDE.md is the case file it reads first.

Exact commands. Not vague verbs.

```
# CLAUDE.md
## Build & test
pnpm test --watch=false
pnpm build
## Directory map
src/api — routes; src/db — migrations; src/ui — components
```

> Not "run the tests." **pnpm test --watch=false** — the exact string it can paste into a shell.

The line that stops a real disaster

```
7  ## Rules
8  Never run db:push against schema.ts.
9  Edits to schema.ts must generate a migration.
```

Never and **must** — not "please be careful with the schema."

Which of these rules will the model actually follow under pressure?

A "Please be careful with the database schema"

B "Never run db:push. Edits to schema.ts must generate a migration."

C "Try to follow good database practices"

COMMIT TO ONE — HANDS UP

Positive suggestions get skimmed.

Negative constraints get obeyed.

LAYER 2 / AGENTS.MD

One file, every tool reads it first

Claude Code | Cursor | Windsurf | GitHub Copilot

Whichever assistant a teammate opens, it checks the same file for the same rules before touching anything.

A specification outlives the chat that produced it

- → **Version-controlled** vs fleeting, gone when the chat closes
- → **Peer-reviewable** vs subjective, lives in one person's head
- → **Persistent** vs localized to one conversation
- → **Comparable across attempts** vs a one-off you can't rerun

CASE STUDY

This is not hypothetical

github.com/vercel/vercel/blob/main/AGENTS.md

```
## Common Pitfalls
- No console.log in CLI packages
- Never skip CI hooks
- Every PR must include a changeset
```

A 247-line file, in a real production monorepo, with a section called "Common Pitfalls."

Plan → Spec → Generate → Validate



Only step 3 is where the model writes code. Steps 1, 2 and 4 are yours — and they're the three that decide whether step 3 is worth trusting.

The job description just changed

Before

The engineer's primary job was generating code.

After

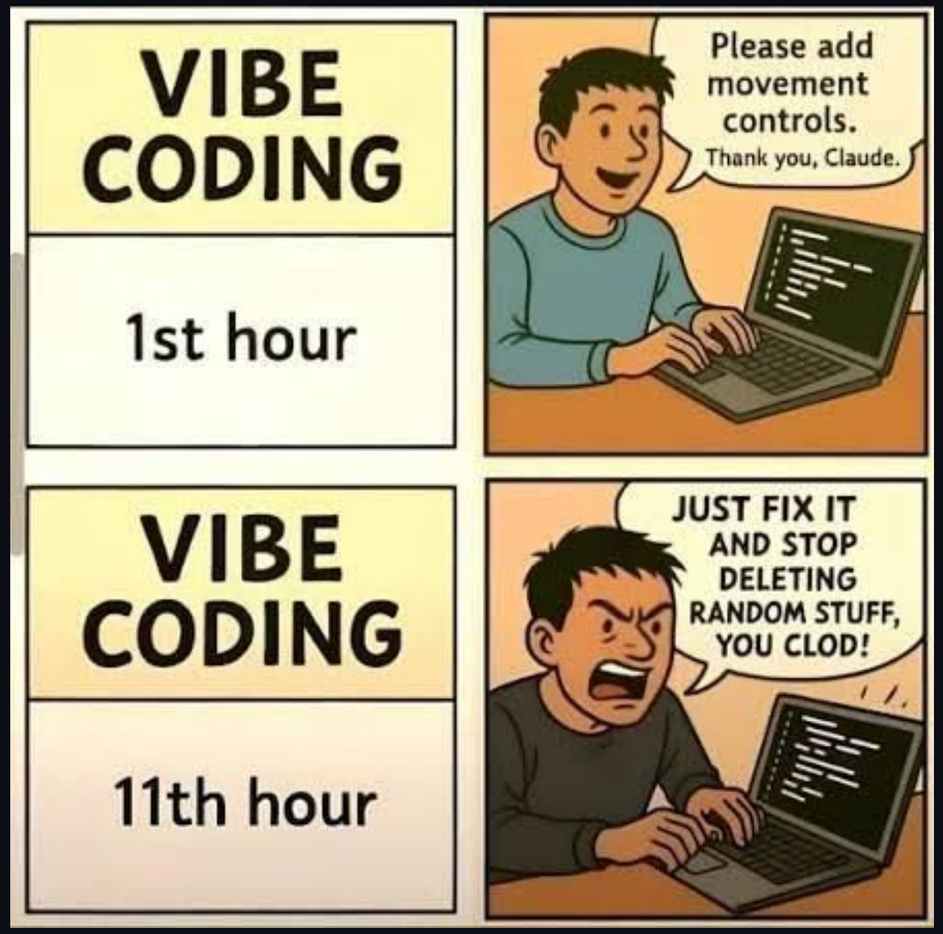
The job is specifying intent precisely, and validating what came back.

What happens to a chat after 50 turns?

It gets worse. Not better.

MEME BREAK

Confidence over 50 turns of context rot



`Working with coding agent`

Turn 1: "here's the one-line fix"

Turn 20: “let me also refactor this"

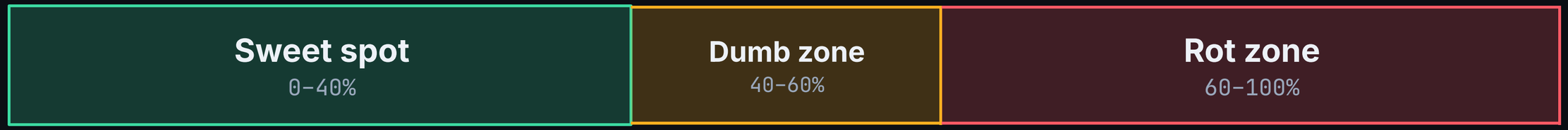
Turn 50: “confidently drops the wrong table”

The fix isn't a bigger context window.

It's a shorter one.

THEORY THREAD

Context got bigger. The "dumb zone" didn't move.



Reliable: instructions followed, retrieval accurate.

Degrading: starts skipping instructions and losing earlier constraints — Dex Horthy (HumanLayer) calls this the "dumb zone."

Unreliable: drift, contradiction, confident hallucination.

The gotcha: the zones are absolute token counts, not clean percentages. A 1M-token window at 40% has already burned **400K raw tokens** — deep in the range where independent testing shows Claude sessions start degrading. A bigger window buys you more room; it does not move the wall.

THEORY THREAD

Every kept token has a running cost

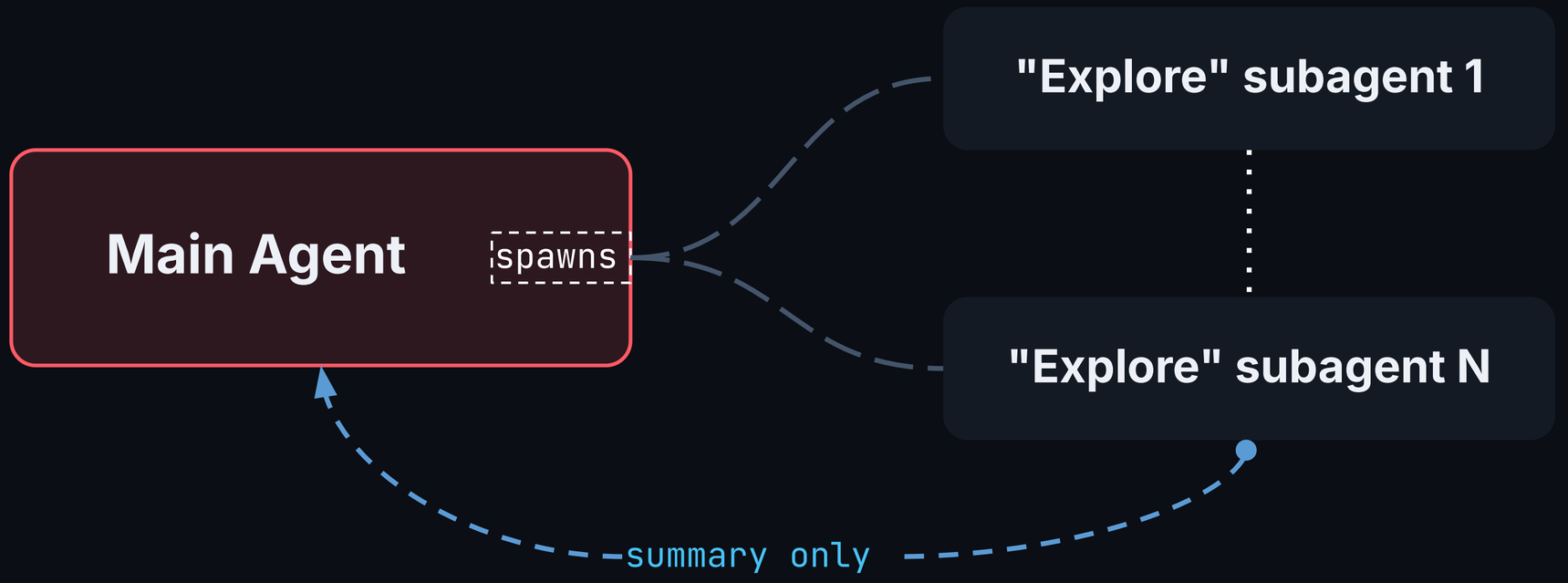
$$cost(n) \propto n^2$$

Attention compares every token to every other token - double the context, quadruple the comparisons.

The KV-cache holding those comparisons grows linearly with every token still resident, whether it's useful or stale.

A 50-turn chat isn't just noisier. It's measurably slower and more expensive per reply.

Read the codebase without polluting the plan



isolated context window — reads the whole repo, nobody upstream sees the noise

The subagent's context is thrown away when it's done. Only its conclusion crosses back.

SKILLS

A Skill is just a script in a folder

```
.claude/skills/pdf-fill/

  SKILL.md          ← name, description, step-by-step instructions
  fill_form.py      ← a script the agent can actually run
  reference.pdf     ← a template it fills in
```

- **Make one:** write SKILL.md, describe when to use it and how, drop it in `.claude/skills/`
- **Install one:** copy someone else's skill folder into the same directory — no registration step
- Claude reads the folder names and descriptions at startup and decides when a task matches one

BEYOND THE CONTEXT FILE

Skills & MCP: validation the agent runs itself

A Skill is a script the agent can invoke on its own — not just generate code, but **run a check and read the result.** MCP is the protocol that connects it to tools outside the chat.

BEYOND THE CONTEXT FILE

Nobody asked it to check this

```
$ skill run check-asset-size demo.gif
checking demo.gif against limit: 3.0 MB
actual size: 4.7 MB
✗ over limit — re-encoding at lower frame rate
✓ demo.gif now 2.1 MB — within limit
```

If it doesn't log to stdout,

the agent is flying blind.

LOGGING AS AN ENABLER

No screen. Only stdout.

```
$ node scripts/send-welcome-email.js --user=412
[mailer] connecting to smtp relay...
[mailer] queued message id=a91f... to user 412
[mailer] delivered in 340ms
```

without that third line, the agent has no way to know the flow finished.

Which of these belongs inside CLAUDE.md?

A The exact test and build commands

B The team's production API keys, for convenience

C Marketing copy about why the product is great

COMMIT TO ONE — HANDS UP

WORKING AS A TEAM

Three buckets, three different fates in git

`committed to git`

Team policy

AGENTS.md / CLAUDE.md,
.claude/settings.json

Build commands, conventions, shared deny rules. Everyone's agent reads the same rules — reviewed in PRs like any other code.

`gitignored, local`

Personal preference

.claude/settings.local.json,
~/.claude/settings.json

Your own auto-approve list, your own editor habits. Never forced on a teammate's machine.

`never a file at all`

Secrets

API keys, DB passwords, tokens

Runtime env vars the process reads at boot — same rule as T1L1. If it's sitting in a file the agent can open, assume it will read it.

SKILLS

Same rule as CLAUDE.md: where it lives decides who gets it

`.claude/skills/ — committed to git`

Team skill

Lives inside the repo. Every teammate gets it the moment they pull — same install-nothing simplicity as AGENTS.md.

`~/.claude/skills/ — your home directory`

Personal skill

Outside any repo, follows you across every project. Good for habits, not for team-specific workflows.

A skill that checks your team's PDF export format belongs in the repo. A skill that formats your commit messages the way you like belongs in your home directory.

WORKING AS A TEAM

Full folder access doesn't mean full read access

.claude/settings.json

```
{
  "permissions": {
    "deny": [
"Read(**/.env*)" ,
"Read(**/secrets/**)" ,
"Read(**/*.pem)"     ]
  }
}
```

The gotcha: a Read deny rule blocks the Read tool. It does not stop `Bash("cat .env")` — same file, different door. Real protection denies both, or runs Bash inside an OS-level sandbox that can't see the path at all.

UPCOMING DELIVERABLES

Build your team's first agent context file

- → Map the directory structure
- → List the exact test and build commands
- → Document project conventions and framework preferences
- → State explicitly what the assistant must **never** touch